~\Desktop\Untitled-1.cpp

```cpp
template<typename T>
class RingBuffer {
    std::vector<T> buffer;
    size_t capacity;
    // alignas(64) 是为了防止“伪共享”(False Sharing)，面试加分项
    alignas(64) std::atomic<size_t> head{0}; // 写位置
    alignas(64) std::atomic<size_t> tail{0}; // 读位置
public:
    RingBuffer(size_t cap) : buffer(cap + 1), capacity(cap + 1) {}
    bool push(const T& item) {// 生产者调用：非阻塞写入
        size_t curr_head = head.load(std::memory_order_relaxed);
        size_t next_head = (curr_head + 1) % capacity;
        // 如果下个位置是 tail，说明满了
        // Acquire 保证读取到最新的 tail
        if (next_head == tail.load(std::memory_order_acquire)) {
            return false;
        }
        buffer[curr_head] = item;
        // Release 保证 item 写入后，head 的更新对消费者可见
        head.store(next_head, std::memory_order_release);
        return true;
    }
    bool pop(T& item) {// 消费者调用：非阻塞读取
        size_t curr_tail = tail.load(std::memory_order_relaxed);
        // Acquire 保证读取到最新的 head
        if (curr_tail == head.load(std::memory_order_acquire))
            return false;
        item = buffer[curr_tail];
        // Release 保证 item 读取后，tail 的更新对生产者可见
        tail.store((curr_tail + 1) % capacity, std::memory_order_release);
        return true;
    }
};
int main() {
    RingBuffer<int> rb(1024);
    std::thread producer([&](){
        for(int i=0; i<100; ++i) {
            while(!rb.push(i)); // 忙等待写入 (Spin)
        }
    });
    std::thread consumer([&](){
        int val;
        for(int i=0; i<100; ++i) {
            while(!rb.pop(val)); // 忙等待读取 (Spin)
            std::cout << val << " ";
        }
    });
    producer.join(); consumer.join();
    return 0;
}
```